From Audio to Insight — Build an AI Podcast Companion
48-Hour Product Innovation Sprint

01 — Research and Context

Why this problem matters

Podcasts are an established and widely consumed medium. According to Edison Research's The UK Podcast Consumer 2025, 71% of UK adults aged 16 and over had ever listened to or watched a podcast, while 51% had consumed one in the previous month.
Research source: Edison Research — The UK Podcast Consumer 2025
Long-form audio can contain valuable ideas, explanations and stories, but listeners may find it difficult to locate specific information, revisit key moments or turn what they hear into actionable knowledge.

AI technologies offer ways to make audio content more accessible and useful through transcription, summarisation, topic extraction, question generation and other forms of content analysis. However, a useful application requires more than a model call: it must handle input reliably, produce meaningful results, manage errors and provide a good user experience. For example, official technical documentation describes approaches to converting recorded speech into text:

- OpenAI — Audio transcription documentation

For API security considerations, see:

- OWASP — API Security Project

These references establish context; they are not a prescribed technical solution.

Your research task

Before building, investigate the problem and decide how you would approach it.

Your research should briefly address:

- **User value:** Who would benefit from an AI podcast companion, and what problem should it solve particularly well?
- **Technology choices:** Which AI models, APIs, libraries or open-source tools could support the workflow? Compare at least two relevant options.
- **Reliability:** How will you reduce inaccurate summaries, unsupported claims and questions that cannot be answered from the source?
- **Practical constraints:** Consider cost, latency, input limits, deployment and maintenance.
- **Responsible handling:** Consider content permissions, uploaded files, API credentials and third-party service requirements.

Use credible sources, prioritising official documentation and relevant technical evidence. Include links to the sources that influenced your decisions and explain what you learned from them. You are expected to conduct your own research and justify your choices. You are not expected to produce a lengthy research report.

02 — The Challenge

Build a small, usable web application that transforms a podcast recording or transcript into an interactive companion that helps people understand, navigate and explore the content. Your application should make it easier for users to identify the main ideas, find relevant sections and test their understanding of an episode. You are free to research, select and combine the AI models, APIs, libraries and development tools you believe are best suited to the problem. We will not prescribe a particular provider, model or technology stack. **The objective is to demonstrate how you independently turn an unfamiliar problem into a working, tested and deployed AI-powered product.**

03 — Core Requirements

1. **Provide content:** Allow a user to submit a text transcript or, if you choose to support it, an audio file. Clearly communicate supported formats, file-size or duration limits, and any other relevant restrictions.
2. **Generate an episode overview:** Produce a concise summary of the supplied content, including its main themes and most important takeaways. The output should be readable, well structured and relevant to the source material.
3. **Create navigable chapters:** Divide the content into meaningful sections with descriptive titles and short explanations. Where reliable timestamps are available or can be derived, include them. If you cannot support timestamps reliably, explain the limitation rather than presenting invented ones.
4. **Generate an interactive learning feature:** Create a short quiz or set of comprehension questions based on the source content. Include answers or a way for users to check their answers. Questions and answers should be grounded in the supplied material rather than invented facts.
5. **Support one meaningful revision:** Let the user request a change to a generated result.

For example, the user might ask for a shorter summary, a simpler explanation or more challenging quiz questions. The revised output should reflect the request without unnecessarily regenerating the entire application workflow.

6. **Deliver a usable result:** Present the generated content in a clear, coherent interface.

Users should be able to review and copy or download the results. Deploy the application so reviewers can access it through a live URL. You may choose how to organise the interface and which features to prioritise, provided the core workflow is functional.

Prioritise a reliable end-to-end experience over a large collection of incomplete features.

04 — Technical Expectations

Your implementation should demonstrate practical AI engineering skills.

- Research and compare at least two relevant tools, providers, models or technical approaches.
- Integrate at least one real AI API or model into the working application.
- Build a sensible workflow for processing input, generating results and handling revisions.
- Validate inputs and outputs where appropriate.
- Handle foreseeable failures, such as invalid input, API errors, timeouts, rate limits or malformed model responses.
- Keep API keys and secrets out of client-side code and public repositories.
- Deploy the application and document any important limitations or external dependencies.
- Provide clear instructions for running the project locally, including configuration requirements.
- Use AI-assisted coding tools if helpful, but be prepared to explain, debug and modify the implementation yourself.

You may use existing libraries and managed services where appropriate. You do not need to train a model from scratch. We are interested in your technical reasoning as much as your final implementation. If a feature cannot be completed within the available time, document the limitation honestly and explain how you would approach it next.

05 — Research and Engineering Notes

Include a concise technical document or README covering the following.

A. Tool and approach comparison: Compare at least two relevant options.

Consider:

- Capabilities and suitability for the task
- Integration complexity
- Output quality and limitations
- Cost, latency or usage constraints
- Why you selected your final approach

Use evidence from your research where possible.

B. System architecture: Explain how your application works, including the main processing stages and how its components interact. A simple architecture diagram is welcome but not required.

C. Debugging example: Describe one meaningful technical issue you encountered.

Explain:

- What went wrong
- How you investigated the problem
- What you changed or tested
- Whether the issue was resolved and what you learned

If you did not encounter a significant issue, identify a failure scenario you deliberately tested and explain the result.

D. Reliability and limitations: Describe important limitations, assumptions and potential failure modes. Explain how you handle—or would handle—incorrect outputs, failed requests, excessive processing time or unexpected inputs.

E. Cost and maintenance: Briefly discuss the likely operating costs, dependencies, configuration and maintenance considerations of your approach. You do not need an exact production budget, but your estimates and assumptions should be transparent.

06 — Deliverables

Submit all of the following:

1. Live application: A working URL reviewers can access and test.

2. Source code: A repository containing your implementation, a README and instructions for running the application locally. Do not include API keys, private credentials or other secrets.

3. Demonstration video: A short screen recording showing the core workflow from input to generated results, including the revision feature.

4. Research and comparison: Your comparison of at least two tools, providers, models or approaches, with links to the sources you consulted.

5. Engineering notes: Your architecture, technical decisions, debugging example, reliability considerations and known limitations.

6. Next steps: Identify the three most important improvements you would make next to enhance functionality, reliability, user experience or scalability. Explain why you would prioritise each one and what impact you expect it to have.

The submission should make it easy for a reviewer to understand what you built, try it, inspect the code and assess your technical reasoning.

07 — Constraints and Expectations

- Use free tiers, trials or approved credits where possible. Do not incur costs without prior approval.
- Do not commit API keys, credentials or other secrets to your repository.
- Use content you have permission to process. Do not upload confidential or sensitive material to third-party services.
- Direct publishing to external platforms is not required.
- You may use AI coding assistants, but you must understand the code and be able to explain your implementation.
- You may choose your own programming language, framework, hosting platform and AI providers.
- You may use existing tools and libraries, provided you can explain their role in the solution.
- Prioritise working functionality, sensible engineering decisions and clear documentation over unnecessary complexity.

Time commitment: The challenge has a 48-hour submission window to provide flexibility around university, work and other commitments. We expect approximately 8–12 hours of effort. Continuous work is neither expected nor encouraged.

08 — Evaluation Criteria

We will evaluate the submission using the following criteria.

- End-to-end functionality and reliability: 25%
- API research, selection and integration: 15%
- Debugging, error handling and resilience: 20%
- Deployment, security and configuration: 15%
- Code quality and maintainability: 10%
- Product judgment and user experience: 10%
- Communication and ownership of decisions: 5%

What strong submissions demonstrate

- A working application rather than only a mock-up or proof of concept.
- Independent research and a clear rationale for technical choices.
- Appropriate integration of real AI capabilities.
- Thoughtful handling of errors and unreliable model outputs.
- Secure handling of credentials and sensible deployment practices.
- Clear, maintainable code and useful documentation.
- Good judgment about what to build within the available time.
- An honest explanation of trade-offs, limitations and future improvements.

We value sound reasoning and reliable execution over the number of features completed.
A short technical discussion may follow submission. You may be asked to explain a design decision, investigate a bug or implement a small change to demonstrate your understanding of the code.

09 — Ownership and Independent Work

Your work remains yours. You retain ownership of the original code, designs and other materials you create for this challenge, subject to applicable law and the terms of any third-party services or licences you use.

Thealon Animation Studios does not claim ownership of your submission simply because you participate in the recruitment process. Participation in this assessment does not, by itself, transfer ownership of your independent prototype to Thealon.

Potential conflicts of interest: If you subsequently join Thealon, any potential overlap between your independent projects and your responsibilities at the studio will be discussed separately. Any work you undertake for Thealon after joining, including work on its products, will be governed by a separate written agreement.

Our intention is to assess your engineering ability, research skills, creativity and problem-solving approach; not to claim ownership of your independent work.

10 — Final Submission Checklist

Before submitting, confirm that you have:

- Deployed the application and tested the live URL.
- Completed the core workflow, including at least one meaningful revision.
- Integrated at least one real AI API or model.
- Compared at least two relevant technical options.
- Included links to credible research and documentation.
- Added the source code and setup instructions.
- Recorded a short demonstration of the application.
- Documented a debugging example, limitations and technical decisions.
- Identified the three most important future improvements.
- Removed credentials and secrets from the repository.

We look forward to seeing how you approach the problem, what you choose to build and how you reason through the engineering challenges